

The Cost of Adaptivity: Online Fanout Selection in Cache-Conscious B-Trees

Anonymous Author(s)

Abstract

B-trees traditionally use a fixed fanout chosen when the index is created. Recent cache-conscious and workload-aware design intuitions suggest a more adaptive alternative: hot regions should use small fanout to improve cache locality, while cold regions should use larger fanout to reduce tree height. This paper asks whether such online fanout adaptation can outperform strong static B-tree baselines on realistic in-memory workloads.

We implement multiple adaptive B-tree variants, including global adaptive trees with reinforcement-learning, regression, and ant-colony-inspired policies, and a segmented adaptive B-tree whose key ranges adapt independently. We compare them against static fanouts, segmented static trees, and oracle baselines that know workload phase boundaries. Our finding is negative but diagnostic: the oracle benefit of changing fanout is usually smaller than the measured cost of monitoring, records metadata, policy checks, and rebuilds. We evaluate a production-grade baseline scaling to 31.7M ops/sec and demonstrate that under rapidly shifting temporal skew, segmented adaptation achieves only 0.63× the throughput of a static baseline, confirming our theoretical break-even model. On five-trial YCSB experiments, segmented adaptation reaches only 0.81×, 0.74×, and 0.67× of static fanout-64 throughput on workloads A, B, and C. On a shifting hot-region workload, oracle fanout selection reaches 1.08×, but segmented adaptation reaches 0.92×. On a Wikipedia pageview trace, static fanout selection has almost no headroom, while adaptation reaches 0.81×. In a separate 1,740-run selected-regime campaign, a retrospective zero-cost oracle reaches 1.029–1.041× static throughput, while the best cost-aware adaptive configuration reaches only 0.881–0.931×; all paired 95% bootstrap intervals exclude parity.

We derive a simple break-even model showing that adaptation can win only when per-operation fanout benefit exceeds monitoring, metadata, and policy cost, plus amortized rebuild cost. The model explains our measurements and provides a practical diagnostic framework for future adaptive index designs.

CCS Concepts

• **Information systems** → **Database performance evaluation**; **Data access methods**; *Database query processing*.

Keywords

B-trees, cache-conscious indexing, adaptive indexing, negative results, workload-aware systems

ACM Reference Format:

Anonymous Author(s). 2027. The Cost of Adaptivity: Online Fanout Selection in Cache-Conscious B-Trees. In *Proceedings of ACM SIGMOD International Conference on Management of Data (SIGMOD '27)*. ACM, New York, NY, USA, 13 pages.

SIGMOD '27, Huntington Beach, CA, USA
2027.

1 Introduction

B-trees remain one of the most important index structures in data management systems [3, 8, 14]. Despite decades of engineering, an apparently simple parameter still matters: fanout. A larger fanout decreases tree height and reduces pointer chasing, but it also creates larger nodes that span more cache lines and increase in-node search cost. A smaller fanout improves locality inside hot nodes but requires more levels. Cache-conscious indexes exploit this trade-off [7, 28, 29], yet most systems choose a single fanout for an entire index.

The adaptive fanout hypothesis is appealing. Real workloads are skewed [9, 39]: a small fraction of keys often receives most accesses. If an index could identify hot regions, it could use small fanout there to keep nodes compact; if it could identify cold regions, it could use large fanout there to reduce height. The resulting tree would be neither globally small nor globally large, but locally matched to access patterns. This paper tests that hypothesis.

We began with the optimistic expectation that online adaptation would improve skewed workloads. Instead, the measurements show a different and, we argue, more useful result. There is measurable oracle headroom: phase-aware fanout choices can improve shifting workloads by roughly 8%. However, online adaptation must pay for monitoring, metadata maintenance, policy evaluation, and rebuilding. In our in-memory setting these costs are comparable to, and often larger than, the entire fanout-selection benefit. The result is that the adaptive variants lose even when the oracle proves that a better fanout exists.

We also stress-test the implementation as a systems artifact. The baseline B+ tree uses optimistic lock coupling and scales from 3.14M ops/sec on one thread to 31.7M ops/sec on 32 worker threads for a one-million-key, ten-million-operation workload. Under rapidly shifting temporal skew, where the hot 20% of keys moves every one million operations, segmented adaptation achieves only 0.63× the throughput of the static baseline. This front-page result is the empirical counterpart of our break-even model: adaptation can be amortized only when fanout benefit exceeds monitoring, metadata, policy, and restructuring costs.

This paper makes five contributions.

- (1) We implement production-style adaptive B-tree variants, including reinforcement-learning, regression, ant-colony-inspired, and segmented adaptive policies.
- (2) We add oracle baselines that quantify the best possible benefit from fanout changes under known phase boundaries.
- (3) We isolate overheads through a clean ablation: static B-tree, static regions, monitoring, records metadata, monitor-plus-records, and full adaptation.
- (4) We report a negative result on YCSB-style workloads, shifting hot regions, and a Wikipedia pageview trace.
- (5) We derive a break-even condition that explains when online fanout adaptation can and cannot win.

Table 1: Allocated key-and-pointer payload per node.

Fanout	Key bytes	Pointer bytes	Total bytes
8	32	72	104
16	64	136	200
32	128	264	392
64	256	520	776
128	512	1,032	1,544
256	1,024	2,056	3,080

Our position is not that fanout is irrelevant. Static fanout choices still matter, and static regional indexes sometimes improve throughput. The diagnostic lesson is narrower: fine-grained online fanout adaptation for in-memory B-trees must be evaluated against oracle headroom and overhead ablations, not merely against a single static default.

2 Background and Motivation

2.1 Fanout and Cache Behavior

A B-tree node with fanout F stores up to $F - 1$ keys and F child pointers. On a 64-bit platform with 32-bit integer keys, fanout 16 creates a small node of a few hundred bytes, while fanout 256 creates a multi-kilobyte node spanning many cache lines. The smaller node may be faster when it is repeatedly accessed and remains in upper cache levels; the larger node may be faster when reducing tree height dominates. Prior cache-conscious B-tree work has explored layout, prefetching, and node sizing [1, 7, 28, 29].

For a tree with N keys, a simplified search cost is

$$C(F, N) = H(F, N)(L + S(F)), \quad H(F, N) = \lceil \log_F N \rceil,$$

where L is the cost of visiting a level and $S(F)$ is the in-node search cost. Changing fanout can improve one term while hurting the other. Because $H(F, N)$ changes only logarithmically, the benefit of switching fanout can be small for moderate in-memory indexes.

Table 1 makes the other side of the trade-off concrete. The implementation allocates a contiguous key array and a contiguous pointer array for each node. Ignoring the fixed node header, allocator metadata, and separately allocated leaf values, the payload is

$$B(F) = F \text{ sizeof(Key)} + (F + 1) \text{ sizeof(void*)}.$$

For the benchmark's 32-bit keys and 64-bit pointers, fanout 256 uses about 3 KiB of array payload, compared with 200 bytes at fanout 16. These values are an analytical footprint, not hardware-counter measurements: they show why fanout can affect locality, but do not by themselves establish where a node is served in the cache hierarchy.

2.2 The Adaptive Hypothesis

Workload-aware data structures have several precedents. They include splay trees [32], adaptive radix trees [22], database cracking and adaptive merging [16, 17], and learned indexes [10, 12, 20]. The analogous hypothesis for B-trees is:

Hot regions should use small fanout for cache locality, while cold regions should use large fanout for height reduction.

The hypothesis is plausible but incomplete. It describes potential benefit but omits the cost of observing workload behavior and changing the index. Our experiments quantify both sides.

2.3 Why Online Fanout Selection Is Difficult

Three effects make the intuitive hot-small/cold-large rule difficult to turn into an end-to-end win. First, tree height changes at discrete thresholds. For a segment containing N_s records, a larger fanout has no height benefit until it removes a level; between thresholds, it can merely increase the amount of data searched inside each node. Second, heat is not the objective. Heat is an observable workload signal, whereas the objective is the latency difference between two complete segment layouts. A concentrated region can be hot while both layouts already fit in memory and therefore have almost identical operation cost. Third, a fanout change requires a physical representation change. The system must retain enough information to build a new segment, determine when evidence is stable, and publish the replacement. These duties remain even for a perfect predictor.

This distinction separates our question from structures that adapt node representation as part of normal insertion, such as ART [22], and from database cracking, whose reorganization work also contributes directly to answering the triggering query [16]. Fanout adaptation adds a control loop to an already functional index. Its benefit is therefore bounded by the gap between two valid layouts, while its observation and actuation costs are additional.

2.4 Evaluation Questions

We organize the study around four questions. **RQ1** asks whether different static fanouts or a phase-aware oracle expose enough headroom to justify adaptation. **RQ2** asks how much of that headroom an online policy captures under stationary and shifting skew. **RQ3** attributes loss to routing, monitoring, V1 shadow records, policy decisions, and rebuilding, and asks how much V2's bounded metadata and bulk loading recover. **RQ4** checks two boundary conditions: whether the underlying B+ tree scales without a global lock, and whether adaptation remains useful when the hot range changes faster than the policy can safely rebuild.

3 Adaptive B-Tree Design

3.1 Architecture and Data Flow

Figure 1 separates the foreground access path and the periodic control path of the original segmented implementation, which we call *Adaptive V1*. Every lookup or update first maps its key to one of eight equal-width ranges and accesses that segment's B+ tree. V1 also maintains a segment-local shadow records map so that a replacement tree can be constructed without scanning an unstable old tree. Sampled accesses update a growing per-key monitor. At a 5,000-access boundary, the control path summarizes heat, concentration, and coverage; predicts a candidate fanout; and applies hysteresis, cooldown, and cost tests. Accepted changes replay the

shadow records into a replacement tree and swap the segment-local tree.

The separation matters experimentally. A cost gate can eliminate an unprofitable rebuild, but it cannot recover the routing, records-map, and sampling work already paid while gathering evidence. Conversely, disabling adaptation while retaining those structures measures the recurring cost that any policy in this implementation family must overcome. This observation motivates the component ladder in Section 4.

3.2 Adaptive V2

Adaptive V2 is a separate implementation variant designed directly from the V1 ablation, not merely a different predictor. It makes three structural changes. First, the B+ tree is the sole record store: inserts and updates do not duplicate values in a shadow hash map, and lookups never fall back to such a map. Second, each segment uses a fixed-size frequency sketch with four rows of 256 counters and a bounded 16-entry heavy-key candidate set. A configurable sample rate (16, 32, or 64 in the selected-regime experiment) limits foreground monitoring work, so monitor memory is independent of the number of distinct keys. Third, once hysteresis, cooldown, and the cost gate accept a change, V2 exports the quiescent segment in key order and constructs the replacement bottom-up with a sorted bulk loader.

V2 retains V1’s eight-range router, candidate fanouts, and conservative control logic so that the comparison isolates representation and observation costs. The current V2 segment-replacement path is single-threaded: export, bulk load, and root replacement occur at a quiescent benchmark point. Consequently, the multi-threaded scalability experiment evaluates only the static B+ tree and is not evidence for concurrent V2 replacement.

3.3 Baseline B+ Tree

Our baseline is a C++11 B+ tree with configurable fanout, supporting insert, update, point lookup, deletion, and range queries. The baseline is deliberately simple enough to isolate fanout effects, yet complete enough to preserve standard B+ tree invariants. The default static configuration uses fanout 64; additional static baselines use fanouts 8, 16, 32, 128, and 256.

3.4 Optimistic Lock Coupling

To make the baseline credible under concurrent access, each node stores an atomic version word. Even versions denote stable nodes; odd versions denote a writer-owned node. Readers do not acquire latches. Instead, they read a node’s version, inspect its keys and child pointer, and validate that the version is unchanged before moving on. A failed validation restarts the lookup from the current root. Writers acquire a node by atomically changing an even version to the following odd version and release it by incrementing the version again. Algorithm 1 shows the lookup path.

The same version word is used for updates. The evaluated scalability benchmark uses a preloaded tree and a 95% read / 5% update workload, so most updates overwrite existing values and acquire only the target leaf. Structural splits acquire the root-to-leaf path in top-down order, publish new nodes, and then release versions in reverse order.

Algorithm 1 Optimistic lock-coupled point lookup

Require: Search key k
Ensure: Found value or NOTFOUND

```

1: repeat
2:    $r \leftarrow$  current root pointer
3:    $n \leftarrow r$ ;  $restart \leftarrow \text{FALSE}$ 
4:   while  $n$  is not a leaf do
5:      $v \leftarrow n.version$ 
6:     if  $v$  is odd then
7:        $restart \leftarrow \text{TRUE}$ ; break
8:     end if
9:      $i \leftarrow$  first slot with  $k < n.keys[i]$ 
10:     $c \leftarrow n.children[i]$ 
11:    if  $n.version \neq v$  then
12:       $restart \leftarrow \text{TRUE}$ ; break
13:    end if
14:     $n \leftarrow c$ 
15:  end while
16:  if not  $restart$  and  $r$  is still the root then
17:     $v \leftarrow n.version$ 
18:    if  $v$  is odd then
19:      continue
20:    end if
21:    scan leaf  $n$  for key  $k$ 
22:    if  $n.version = v$  then
23:      return matching value or NOTFOUND
24:    end if
25:  end if
26: until lookup validates

```

3.5 Monitoring

The monitor tracks approximate heat for nodes or key ranges. The segmented tree records key identifiers, so the resulting summary describes access concentration inside a segment rather than physical-node residency. To limit overhead, the outer segmented tree forwards one in ten accesses to the monitor. The monitor uses 64 metadata shards, sampled access counts, lazy decay with factor 0.99, and an exponential moving average:

$$h_{t+1} = \alpha a_t + (1 - \alpha)h_t.$$

Here $a_t = 1$ for a sampled touch and $\alpha = 0.1$; decay is applied when a tracked identifier is next inspected or touched. The monitor exposes tracked identifiers, decay, and aggregate statistics. It is intentionally lightweight, but it still increments a segment counter on every operation and occasionally performs a hash-table lookup, timestamp read, lock acquisition, and metadata update. Its map also grows with the distinct sampled keys in the finite benchmark run. The ablation therefore reports both throughput and the number of monitored identifiers instead of treating monitoring as free.

3.6 Prediction Policies

We implement three global policies. A Q-learning policy maps coarse heat states to actions that decrease, maintain, or increase fanout. A regression policy predicts fanout from heat and access count. An ant-colony-inspired policy maintains pheromone scores

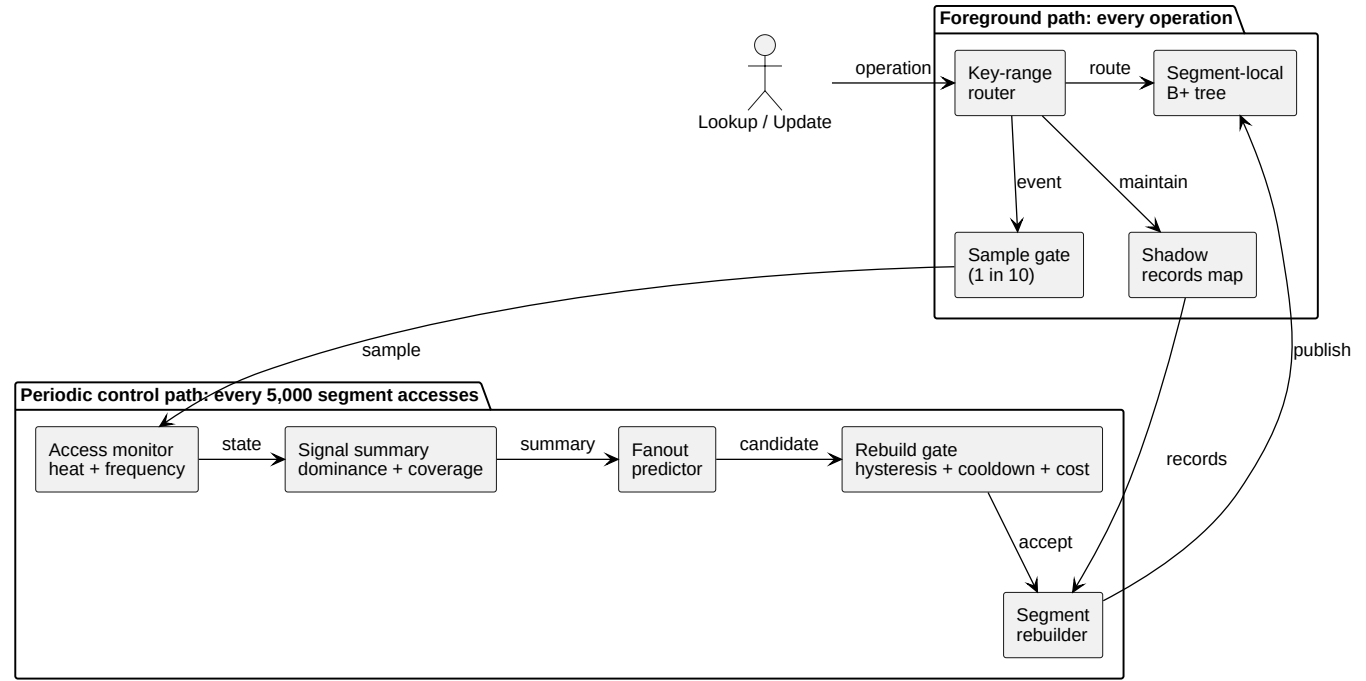


Figure 1: Architecture of Adaptive V1. Routing, shadow-record maintenance, and the sample gate are foreground work. Every 5,000 segment accesses, the periodic path may build and publish a replacement tree after prediction and cost checks.

over heat/action pairs. These policies are included not because we expect them all to win, but to test whether the result depends on policy choice. In our measurements, policy sophistication is not the main limiter; the common monitoring and metadata path is.

3.7 Segmented Adaptation

The most important adaptive design is segmented. It partitions the key space into fixed ranges. Each segment owns its own B+ tree, monitor, records map, and adaptation state. Segment-level rebuilds are less disruptive than global rebuilds and match shifting hot-region workloads better. The segmented design also enables clean ablations because we can disable records, monitoring, and adaptation independently.

For a key domain $[K_{\min}, K_{\max}]$ and S segments, routing uses integer arithmetic to compute

$$s(k) = \min\left(S - 1, \left\lfloor \frac{(k - K_{\min})S}{K_{\max} - K_{\min} + 1} \right\rfloor\right).$$

This is $O(1)$, but it adds bounds checks, an indirection through the segment array, and a second ownership boundary before every tree operation. More importantly, the records map is not a few bytes of policy metadata: it is a complete key/value image of the segment. Maintaining it duplicates logical records and adds a hash-table update on writes. We retain this design because it makes reconstruction explicit and safe in the single-threaded adaptive experiments; Section 7 discusses alternatives.

3.8 Cost-Aware Rebuild Gate

The adaptive tree does not rebuild merely because a predictor recommends a new fanout. A cost-aware gate rejects rebuilds when the expected benefit is too small, when evidence is weak, or when a segment is inside a cooldown period. This gate reduces unnecessary rebuilds, but it does not remove ongoing access path costs.

Algorithm 2 gives the exact decision flow used by the segmented tree. The policy first summarizes the sampled monitor state, then requires directional stability through repeated votes. A segment is rebuilt only if the cooldown has expired and the estimated benefit exceeds estimated rebuild cost. This is deliberately conservative: rejected rebuilds are counted and reported as cost skips in the experiments.

The concrete gate requires at least 20 samples. A fanout jump of at least $4\times$ needs two consistent votes; a smaller change needs three. After a rebuild, a segment cannot rebuild again for four adaptation intervals, or 20,000 segment accesses in the reported configuration. Shrinking additionally requires heat of at least 0.68 and either an 8% dominant-key fraction or a 15% hot-key fraction. Growing requires the segment to have started below fanout 64, heat at most 0.35, at least 16 tracked keys, and a dominant-key fraction below 2%. These thresholds deliberately bias the system toward *not* rebuilding; they reduce catastrophic oscillation while exposing the recurring cost of gathering enough evidence to reject a change.

Rebuild complexity. V1 constructs an empty B+ tree at fanout f' and inserts every entry from the segment's shadow records map. For n_s records this costs $O(n_s \log_{f'} n_s)$ comparisons and allocations under ordinary insertion. V2 instead exports the segment's leaves

Algorithm 2 Cost-aware fanout rebuild gate

Require: Segment s , sampled monitor state, current fanout f
Ensure: Either rebuild to f' or defer adaptation

```

1:  $H \leftarrow$  aggregate heat score of sampled keys in  $s$ 
2:  $D \leftarrow$  dominant access fraction in  $s$ 
3:  $U \leftarrow$  number of sampled unique keys/nodes
4:  $f_p \leftarrow \text{PREDICTFANOUT}(H, D, U)$ 
5:  $f' \leftarrow \text{CLAMPTOPOLICYBANDS}(f_p, H, D)$ 
6: if  $f' = f$  then
7:   count maintained fanout; defer next check
8: return
9: end if
10: if  $f'$  changes direction relative to pending vote then
11:   reset pending target to  $f'$  with one vote
12: else
13:   increment vote count for  $f'$ 
14: end if
15: if votes for  $f'$  are below hysteresis threshold then
16:   count hysteresis skip; defer next check
17: return
18: end if
19: if segment is inside rebuild cooldown then
20:   count cooldown skip; defer until cooldown expires
21: return
22: end if
23:  $B \leftarrow \text{ESTIMATEBENEFIT}(s, f, f', H, D, U)$ 
24:  $C \leftarrow \text{ESTIMATEREBUILDCOST}(s)$ 
25: if  $B \leq C$  then
26:   count cost skip; defer next check
27: return
28: end if
29: rebuild segment records into a new B+ tree with fanout  $f'$ 
30: publish rebuilt segment; reset pending votes

```

in sorted order and builds leaf and internal levels bottom-up in $O(n_s)$ time and space. This removes repeated searches and reduces allocation count, but it does not make a fanout change constant time: export and construction still copy the complete segment. The gate uses a conservative linear rebuild score for V2 and an $n_s(1 + \log_2 n_s)$ score for V1; both are engineering heuristics rather than calibrated nanosecond predictors. Their purpose is to reject clearly unprofitable changes and report the rejection reason.

4 Experimental Methodology

4.1 Implementation and Platform

The prototype is implemented in C++11 and compiled with GCC 6.3.0 under Windows/MinGW. The codebase contains roughly 2700 lines across the baseline tree, monitor, predictor, adaptive policies, segmented tree, tests, and benchmarks. Experiments run on one Windows 11 machine with a 12th-generation Intel Core i5-12500H (12 physical cores, 16 hardware threads) and 16 GiB of RAM. The GPU is not used. This machine is intentionally modest: the goal is to study main-memory index overhead, not to rely on a large server or an accelerator.

Table 2: Principal experimental parameters.

Parameter	Setting
Language / compiler	C++11 / GCC 6.3.0
Machine	i5-12500H, 16 threads
Memory / OS	16 GiB / Windows 11
Default / candidate fanouts	64 / 8–256
Segments / sampling	8 / V1: 1/10; V2: 1/16–1/64
Adaptation interval	5,000 accesses
YCSB records / operations	100K / 100K
YCSB Zipf parameter	0.99
Shifting operations / phase	30K
Repeated trials	5
Thread records / operations	1M / 10M
Sliding windows	$5 \times 1\text{M}$ operations

Table 2 lists the principal settings. The stationary experiments use 100K records so all variants fit comfortably in memory; the thread and sliding-hotspot experiments use one million records to exercise a larger index while remaining practical on the same machine. Unless noted, the initial fanout is 64, the key domain is divided into eight equal ranges, and adaptation is considered every 5,000 accesses to a segment.

4.2 Compared Indexes and Oracles

The baseline family contains a monolithic static B+ tree at fanout 64 and 256, plus a static regional index with eight independently allocated trees. The shifting experiment sweeps regional fanout over $\{8, 16, 32, 64, 128, 256\}$. The adaptive family contains global RL, regression, and ant-colony-inspired policies; a segmented tree with adaptation disabled; Adaptive V1 with growing monitoring and shadow records; and Adaptive V2 with bounded monitoring, no shadow map, and sorted bulk loading. These are not claimed as state-of-the-art learned-index competitors. They test whether changing both policy sophistication and implementation cost alters the dominant cost conclusion within one common B+ tree implementation.

The phase oracle is a diagnostic upper bound. It knows the next phase boundary and hot segment, rebuilds that segment at fanout 8, 16, or 32, and reports both operation-only throughput and throughput including measured reconfiguration time. A second retrospective envelope chooses the best measured static regional fanout for each phase. The oracle answers RQ1: if advance knowledge cannot beat a static baseline, an online detector has no opportunity to capture. It is not presented as an implementable competitor.

The artifact additionally defines *Perfect-V1* and *Perfect-V2* configurations for a future decomposition. They receive the selected fanout and phase boundary from a measured phase-oracle file but still pay their variant’s real reconstruction cost. This separates detection error from actuation cost. We do not report those rows in the present paper because that campaign has not yet been executed; the retrospective zero-cost oracle used in Section 5.7 remains the deliberately optimistic opportunity bound.

4.3 Workloads

We evaluate six workload families.

YCSB-style workloads. Workload A is 50% reads and 50% updates, workload B is 95% reads and 5% updates, and workload C is read-only. Keys follow a Zipfian distribution with $\theta = 0.99$, matching the skewed access pattern used in many storage-system studies [9].

Shifting hot regions. Four phases target segment 0, segment 7, the full key domain uniformly, and segment 3. Each phase contains 30K operations. This workload favors adaptation because the profitable regional fanout can change over time, and the oracle receives the phase boundary and hot segment.

Wikipedia pageview trace. We replay 30K operations on each of ten consecutive days of a trace built from top Wikipedia articles using Wikimedia’s pageview data [36]. The trace is highly skewed and supplies a real popularity distribution, although it is not a full database transaction trace.

Overhead ablation. We run a seven-row ladder: Static B+Tree, Static Region, Static Region + Monitor, Segmented No Records, Segmented Records Only, Segmented Monitor + Records, and Segmented Full Adaptive. The ladder isolates routing, shadow-record maintenance, monitoring, and adaptive decision/rebuild costs without assuming every adjacent difference is statistically separable.

Concurrent workload. A preloaded one-million-key tree executes ten million operations with 95% reads and 5% in-place updates at 1, 4, 8, 16, and 32 worker threads. This workload evaluates the OLC baseline, not concurrent segment replacement.

Sliding hotspot. Five one-million-operation windows each direct 80% of accesses to a disjoint 20% range of a one-million-key domain. The operation mix is 95% reads and 5% updates. This tests whether the control loop safely reacts when old evidence becomes stale at every window boundary.

4.4 Measurement and Statistical Treatment

For each workload, the operation sequence is generated once from a fixed seed and replayed across all index variants. Index construction is timed separately and excluded from operation throughput. Throughput is total completed operations divided by wall-clock run time. YCSB records every operation latency; the larger sliding-hotspot benchmark samples one latency per 1,024 operations to bound instrumentation cost. A checksum and miss count prevent dead-code elimination and detect semantic disagreement.

YCSB, shifting-hot-region, and overhead-ablation experiments are each repeated five times. We report arithmetic means, sample standard deviations, and two-sided 95% confidence intervals using Student’s t distribution with four degrees of freedom. The thread, initial sliding-hotspot, and ten-day Wikipedia experiments are descriptive single campaigns and are not assigned trial-to-trial confidence intervals. We therefore emphasize large, cross-workload effects and disclose high-variance cases instead of treating small ordering differences as significant.

We additionally run a selected-regime validation on a four-vCPU Kaggle CPU instance using GCC 11.4.0, one million records, and one million measured operations per run. The campaign contains 1,740 completed runs and no failed or partial runs. Adaptive/static comparisons match seed and repetition; 95% hierarchical bootstrap

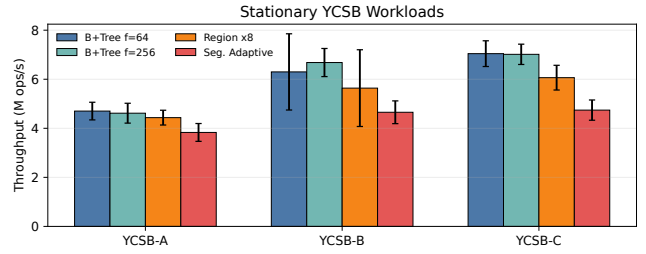


Figure 2: Five-trial YCSB throughput. Error bars show 95% confidence intervals. Segmented adaptation is consistently below static fanout 64.

Table 3: Five-trial YCSB throughput summary.

Workload	Static f64	Best static	Seg. adaptive	Seg./f64
YCSB-A	4.70M	4.70M	3.83M	0.81
YCSB-B	6.30M	6.68M	4.65M	0.74
YCSB-C	7.04M	7.04M	4.74M	0.67

intervals resample the five workload seeds and then the ten repetitions within each seed. Oracle comparisons match each seed to its static calibration. Because this campaign uses a different machine, we report within-machine ratios and do not pool absolute throughput across hosts.

5 Results

5.1 Stationary YCSB

Figure 2 and Table 3 show the YCSB results. Segmented adaptation loses on all three workloads. On YCSB-A it reaches 0.81× of static fanout 64. On YCSB-B it reaches 0.74×; on YCSB-C it reaches 0.67×. Static fanout 256 wins YCSB-B by roughly 6%, showing that fanout choice matters, but the adaptive path is too costly to capture this gain.

The uncertainty supports the same conclusion with one caveat. On YCSB-A, static fanout 64 is $4.70\text{M} \pm 0.36\text{M}$ operations/s and segmented adaptation is $3.83\text{M} \pm 0.36\text{M}$; their 95% intervals are separated. On YCSB-C the intervals are also separated ($7.04\text{M} \pm 0.52\text{M}$ versus $4.74\text{M} \pm 0.41\text{M}$). Fanout-64 YCSB-B is noisy, with a 19.9% coefficient of variation, so its interval overlaps the adaptive result. The stronger static fanout-256 comparison is $6.68\text{M} \pm 0.57\text{M}$ versus $4.65\text{M} \pm 0.46\text{M}$, whose intervals do not overlap. We therefore do not claim that every static ordering is significant; we claim that the best static configuration is consistently faster than online segmentation.

Policy choice does not rescue the global design. Across YCSB-A/B/C, the ant-colony-inspired tree reaches only 0.43/0.34/0.33 of fanout-64 throughput, regression reaches 0.28/0.23/0.21, and RL reaches 0.20/0.16/0.16. The segmented policy is substantially better because it localizes rebuilds, yet it still loses. This ordering points away from prediction accuracy as the sole problem: all global variants share expensive whole-tree adaptation, while the best adaptive variant still pays the common foreground path.

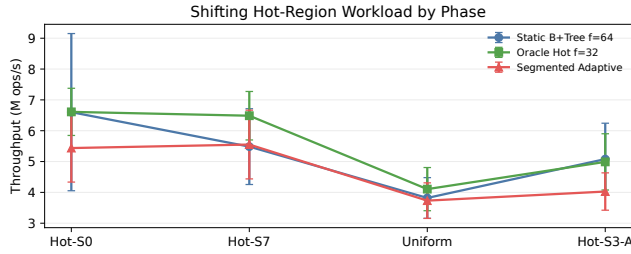


Figure 3: Shifting hot-region throughput by phase. The oracle shows measurable headroom, but online segmented adaptation does not capture it.

Table 4: Aggregate shifting workload throughput.

Index	Throughput	vs f64
Static Region f256	5.53M	1.13
Oracle Hot f32	5.29M	1.08
Static B+Tree f256	5.24M	1.07
Static B+Tree f64	4.86M	1.00
Segmented Adaptive	4.49M	0.92

5.2 Shifting Hot Regions

The shifting workload gives adaptation a fairer opportunity because locality changes over time. Figure 3 plots selected per-phase results. The oracle with hot fanout 32 reaches 1.08× aggregate throughput over static fanout 64, confirming that fanout adaptation has headroom. However, segmented adaptation reaches only 0.92×. Static regional fanout 256 reaches 1.13×, showing that coarse static partitioning can be more effective than online adaptation.

The phase breakdown also warns against selecting a single favorable phase. Static regional fanout 256 reaches 1.35× fanout 64 in Hot-S7 but only 0.93× in Hot-S0. Oracle-hot fanout 32 ranges from 0.98× to 1.18× across measured phases. Thus locality does change the preferred layout, satisfying the premise of adaptation, but the phase samples are variable: several static rows have coefficients of variation above 20%. We use the five-trial aggregate and the oracle/static envelopes as directional evidence rather than claiming a universally optimal regional fanout.

5.3 Wikipedia Pageview Trace

Table 5 shows the real-trace result. Static fanout 256 and fanout 64 are nearly identical: 10.42M and 10.39M operations/s. Thus the workload has almost no fanout-selection headroom at this scale. Segmented adaptation reaches 8.40M operations/s, or 0.81× of static fanout 64. The trace is skewed, but the hot working set is small enough that static fanout choice is not the limiting factor.

The ten daily runs make the absence of headroom visible in another way. Fanout-64 daily throughput ranges from roughly 9.18M to 11.20M operations/s, whereas fanout-256 ranges from 10.01M to 10.75M. Their aggregate means differ by only about 0.3%, smaller than the variation across days. The adaptive tree nevertheless performs 37 policy evaluations and one rebuild over the trace. Extreme popularity skew therefore does not imply a useful

Table 5: Wikipedia pageview trace throughput.

Index	Throughput	vs f64
Static B+Tree f256	10.42M	1.00
Static B+Tree f64	10.39M	1.00
Static Region f32	10.19M	0.98
Static Region x8	9.88M	0.95
Segmented No Adapt	8.79M	0.85
Segmented Adaptive	8.40M	0.81

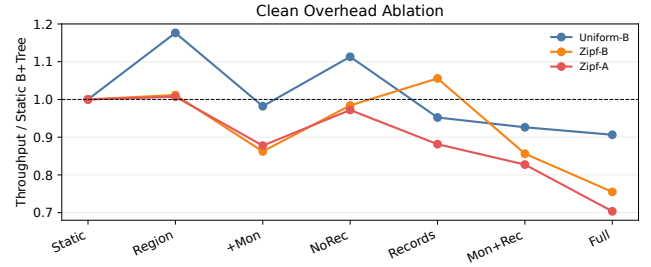


Figure 4: Clean overhead ablation normalized to Static B+Tree. The full adaptive path includes routing, records, monitoring, policy checks, and rebuild decisions.

fanout signal: once the active set is already cache-resident, extra heat tracking can remain pure overhead.

5.4 Clean Overhead Ablation

Figure 4 and Table 6 isolate the access-path costs. The values are intentionally not monotone in every row: cache placement, routing, and run-to-run noise can make one row faster than the previous row. The robust pattern is that the full adaptive path is below static on Zipf-A and Zipf-B and below the best static regional rows. In Zipf-A, the records-only row reaches 4.90M operations/s, the monitor-plus-records row reaches 4.60M operations/s, and full adaptation reaches 3.91M operations/s.

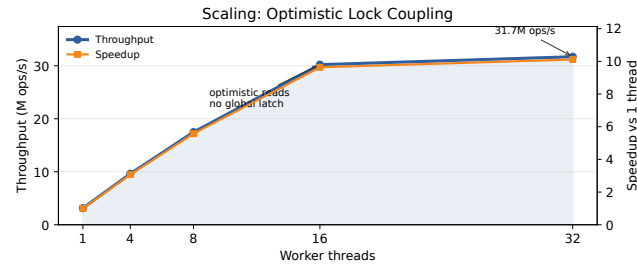
Repeated trials make two conclusions defensible. First, individual adjacent rows are not always ordered: for example, records-only is faster than the no-records row on Zipf-B. We treat such inversions as cache placement and run-order noise, not as negative-cost metadata. Second, the end-to-end gaps are much larger. On Zipf-A, the lower endpoint for Static B+Tree is 4.57M operations/s, while the upper endpoint for Full Adaptive is 4.21M. On Zipf-B the corresponding endpoints are 6.52M and 6.10M. Uniform-B intervals overlap, so we make no significance claim there. The controlled conclusion for RQ3 is that monitoring and decisions are first-order costs under skew, while no single neighboring row should be over-interpreted.

5.5 Concurrent Lookup Scalability

The original prototype was single-threaded. To test whether the baseline can serve as a credible systems artifact, we added optimistic lock coupling to the B+ tree. Each node stores an atomic version word; readers validate versions before and after traversal,

Table 6: Clean overhead ablation. Ratios are relative to the previous conceptual baseline named in each column.

Workload	Region/Static	Monitor/Region	NoRec/Region	Rec/NoRec	MonRec/Rec	Full/MonRec	Full throughput
Uniform-B	1.18	0.83	0.95	0.86	0.97	0.98	4.79M $\pm$ 0.41M
Zipf-A	1.01	0.87	0.97	0.91	0.94	0.85	3.91M $\pm$ 0.30M
Zipf-B	1.01	0.85	0.97	1.07	0.81	0.88	5.68M $\pm$ 0.42M

**Figure 5: Optimistic lock coupling scalability on a one-million-key tree and ten-million-operation workload.****Table 7: Thread-scalability benchmark, 95% reads and 5% updates.**

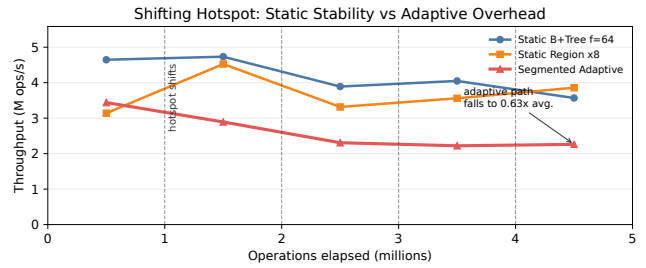
Threads	Throughput	Speedup
1	3.14M	1.00
4	9.63M	3.07
8	17.48M	5.57
16	30.21M	9.64
32	31.70M	10.11

while writers acquire per-node version locks. The thread benchmark preloads one million keys and executes ten million operations with 95% reads and 5% updates. Figure 5 and Table 7 show that the baseline scales to the 16 hardware threads of our i5-12500H and flattens under oversubscription at 32 worker threads. This result does not reverse the adaptive-index conclusion, but it removes a major systems concern: the static baseline is not protected by a single global lock.

Scaling is sublinear but substantial: 16 threads deliver 9.64 $\times$ speedup, or 60% parallel efficiency, and 32 workers reach 10.11 $\times$. The small increase from 16 to 32 is consistent with running twice as many workers as hardware threads and with contention from the 5% update share. Because this is a single campaign, the exact peak should not be read as a population estimate; its role is architectural validation that readers traverse without a global latch.

5.6 Rapidly Shifting Hotspots

We also add a sliding-hotspot benchmark. The workload contains one million keys and five one-million-operation windows. In each window, 80% of accesses target a Zipfian hot region covering 20% of the key space; the hot region moves to a disjoint range in the next window. This benchmark is intentionally hostile to stale predictors. Figure 6 and Table 8 show that the adaptive tree does not catch up:

**Figure 6: Sliding-hotspot throughput by one-million-operation window. The adaptive tree tracks accesses but remains below static fanout 64.****Table 8: Sliding-hotspot aggregate throughput.**

Index	Throughput	vs Static f64
Static B+Tree f64	4.18M	1.00
Static Region x8	3.68M	0.88
Segmented Adaptive	2.62M	0.63

the cost gate evaluates many adaptation opportunities but performs no rebuilds, and the full segmented adaptive path reaches only 0.63 $\times$ the static B+ tree throughput. This strengthens the paper’s negative result because the workload gives adaptation fresh skew but too little safe headroom to exploit.

The rejection counters reveal why. Across the five windows, the policy runs 264 adaptation evaluations, rejects 247 at the cost gate, and performs zero rebuilds. Hence the controller is not losing because it repeatedly rebuilds the wrong segment. It is losing while correctly deciding that available evidence does not justify reconstruction. The first adaptive window reaches 3.44M operations/s, but later windows fall to 2.22–2.89M as monitored state accumulates; fanout remains 64 throughout. This is a particularly strong negative case for fine-grained control: conservative actuation avoids rebuild spikes, yet observation and shadow-state maintenance still consume 37% of the static baseline’s throughput.

5.7 Selected-Regime Validation

The preceding sliding-hotspot experiment establishes the failure mode, but a single trajectory cannot determine whether the result is stable across sampling rates and workload seeds. We therefore use a measured static-fanout screening campaign to select three regimes: a long, highly skewed regime with little expected headroom (Zero); a long regime near the selection boundary (Borderline); and a rapidly changing regime with the largest measured

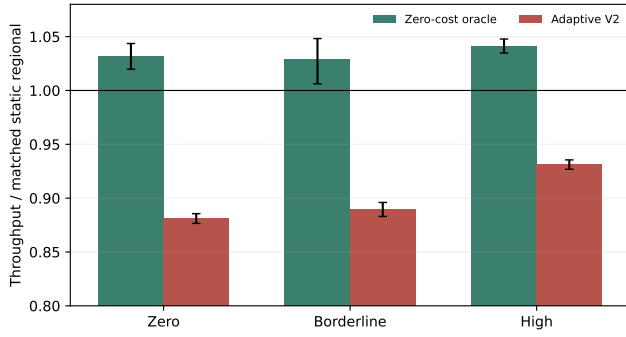


Figure 7: Selected-regime validation with paired 95% bootstrap intervals. A retrospective zero-cost oracle is above static, while Adaptive V2 is significantly below its matched static-regional baseline in every regime.

Table 9: Selected-regime validation. Adaptive columns use 50 paired runs; oracle ratios use five matched seeds.

Regime	Oracle/static	Adapt./static	95% CI	Penalty ($\mu\text{s/op}$)
Zero	1.032	0.881	[0.877, 0.886]	0.0211
Borderline	1.029	0.890	[0.883, 0.896]	0.0219
High	1.041	0.931	[0.927, 0.936]	0.0199

headroom (High). For each regime we evaluate Adaptive V1 and the cost-aware Adaptive V2 at sample rates 16, 32, and 64. Each cohort contains five seeds and ten repetitions per seed. Separate static-fanout calibration runs construct a retrospective, zero-cost phase oracle on the same machine and binary.

Figure 7 and Table 9 report seed/repetition-matched ratios. The best V2 setting is sample rate 64 in all three regimes. Nevertheless, V2 reaches only 0.881 $\times$, 0.890 $\times$, and 0.931 $\times$ of static regional throughput. Every 95% interval lies below one. The zero-cost oracle confirms genuine but narrow opportunity: 1.032 $\times$, 1.029 $\times$, and 1.041 $\times$, respectively. Thus the conclusion is not that fanout never matters; it is that this online mechanism cannot capture a benefit of this size cheaply enough.

The time-domain comparison in Figure 8 makes the break-even failure explicit. Across the three regimes, the oracle saves 0.0049, 0.0050, and 0.0106 $\mu\text{s/op}$, whereas V2 adds 0.0211, 0.0219, and 0.0199 $\mu\text{s/op}$ relative to its paired static baseline. Even in the High regime, ongoing adaptive cost is about 1.9 $\times$ the maximum retrospective benefit before charging any uncertainty or prediction error. Longer phases cannot repair the first two regimes because their recurring cost already exceeds their per-operation benefit.

5.8 Cross-Workload Synthesis

Table 10 answers RQ1 and RQ2 in one view. The opportunity column is the best measured static configuration for stationary workloads and the phase-aware oracle for the shifting workload, normalized to static fanout 64. It is deliberately generous to the adaptive hypothesis. YCSB-A, YCSB-C, and Wikipedia expose essentially no selection headroom; online adaptation cannot win there unless it is free. YCSB-B and shifting do expose 6–8% headroom, but segmented

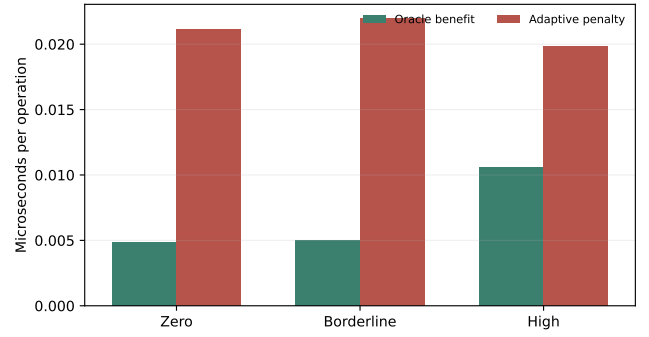


Figure 8: Oracle benefit and Adaptive V2 penalty in time per operation. The online penalty exceeds the available fanout benefit in every regime.

adaptation gives up 26% and 8% relative to fanout 64. The selected-regime campaign independently reproduces the same pattern: even its High regime offers only 4.1% oracle headroom while Adaptive V2 gives up 6.9%. The headroom is real, but smaller than the cost of finding and acting on it.

The table also separates two kinds of negative result. In a *no-opportunity* workload, the right response is an early bypass that turns monitoring off. In an *uncaptured-opportunity* workload, the research problem is to reduce observation and reconstruction cost below the measured oracle gap. A policy paper that reports only final throughput would conflate these cases; oracle and ablation rows make the diagnosis actionable.

5.9 Memory Reclamation and Reader Safety

Optimistic reads raise a standard systems question: if readers traverse without latches, when is it safe to reclaim old nodes? The implementation used for the concurrency benchmark avoids unsafe reclamation in two ways. First, the benchmark preloads the tree and then executes lookups and in-place updates; updates overwrite existing values and do not delete nodes. Second, structural splits are publication-only: a split allocates a new node and links it into the tree, while the old node remains valid. The tree is destroyed only after all worker threads have joined. Thus a reader that validates an old version can safely retry without dereferencing freed memory.

Segment rebuilds for adaptive fanout are more subtle. In the current experiments, segmented rebuilds are evaluated in the single-threaded adaptive benchmarks, not concurrently with the multi-threaded lookup benchmark. A fully concurrent adaptive tree should retire replaced segment roots using epoch-based reclamation: each worker publishes its active epoch before traversal, writers place old roots on a retire list, and a retired tree is freed only after all active readers have advanced beyond the retire epoch. This is an engineering constraint rather than a change to the break-even model; epoch management would add another small term to the online overhead μ .

6 Theoretical Analysis

We now formalize the cost argument. The theory has two roles. First, it shows that restructuring itself can be amortized if the algorithm

Table 10: Opportunity versus achieved segmented-adaptive throughput. Values are normalized to static fanout 64.

Workload	Opportunity	Seg. adaptive	Gap to opportunity	Interpretation
YCSB-A	1.00	0.81	-0.19	No static headroom; recurring cost dominates
YCSB-B	1.06	0.74	-0.32	Headroom exists but is not captured
YCSB-C	1.00	0.67	-0.33	Read-only path magnifies overhead
Shifting hot regions	1.08	0.92	-0.16	Oracle wins; online control still loses
Wikipedia trace	1.00	0.81	-0.19	Skew without useful fanout separation
Sliding hotspot	—	0.63	—	Gate rejects all rebuilds, but tracking remains
Selected high-headroom	1.041	0.931	-0.110	Paired CI excludes parity

waits for enough evidence. Second, it shows why that amortization is insufficient when per-operation monitoring and metadata costs exceed the fanout benefit.

6.1 Amortized Restructuring

Let a segment maintain a nonnegative potential Φ_t . Each operation adds at most a constant γ units of potential. A rebuild of cost R_t is allowed only when $\Phi_t \geq R_t$, and when it rebuilds the algorithm spends R_t units of potential.

THEOREM 1. *If the rebuild rule above is enforced, restructuring has $O(1)$ amortized cost per operation.*

PROOF. Let a_t be the actual restructuring cost at operation t , and define the amortized cost

$$\hat{a}_t = a_t + \Phi_{t+1} - \Phi_t.$$

If no rebuild occurs, $a_t = 0$ and $\Phi_{t+1} - \Phi_t \leq \gamma$, so $\hat{a}_t \leq \gamma$. If a rebuild occurs, then $a_t = R_t$ and the potential decreases by at least R_t , except for the current operation's new credit:

$$\Phi_{t+1} - \Phi_t \leq \gamma - R_t.$$

Thus $\hat{a}_t \leq R_t + \gamma - R_t = \gamma$. Summing over T operations telescopes the potential terms, giving total restructuring cost $O(T)$, or $O(1)$ amortized cost per operation. $\square$

6.2 A Conditional Online Bound

A multiplicative competitive claim requires assumptions about exploration; otherwise an arbitrarily bad candidate fanout can make a testing epoch arbitrarily expensive. We therefore state the finite-candidate bound with those assumptions explicit. Consider a stable phase P of T_P operations and candidate set $\mathcal{F}$ of size k . Let c_P^* be the oracle operation cost in the phase. The online policy spends at most q operations testing each candidate, all tested candidates cost at most βc_P^* , and its selected candidate costs at most αc_P^* . Let μ be recurring online overhead and R_P the total restructuring cost in the phase.

THEOREM 2. *Under the conditions above, the phase cost of the online policy satisfies*

$$\frac{\text{Cost}_P(A)}{\text{Cost}_P(\text{OPT})} \leq \alpha + \frac{kq(\beta - \alpha)}{T_P} + \frac{\mu}{c_P^*} + \frac{R_P}{T_P c_P^*}.$$

PROOF. At most kq operations are exploratory and cost at most βc_P^* each. The remaining $T_P - kq$ operations use the selected candidate at cost at most αc_P^* . Adding recurring and restructuring costs

gives

$$\text{Cost}_P(A) \leq kq\beta c_P^* + (T_P - kq)\alpha c_P^* + T_P\mu + R_P.$$

Dividing by $T_P c_P^* = \text{Cost}_P(\text{OPT})$ yields the claim. $\square$

For a double-exponential theoretical ladder, $k = O(\log \log n)$. The implementation instead uses the constant candidate set

$$\mathcal{F} = \{8, 16, 32, 64, 128, 256\}.$$

With fixed q , the exploration and rebuild terms shrink in a long stable phase. The recurring ratio μ/c_P^* does not. This is the formal reason that “wait longer” cannot repair the implementation when monitoring and shadow-state work already exceed the oracle benefit.

6.3 Break-Even Condition

The measurements can be explained by a simple condition. Consider a phase of T operations in a segment with incumbent fanout F_0 and candidate fanout F_1 . Let

$$\Delta = C(F_0, N_s) - C(F_1, N_s)$$

be the per-operation fanout benefit. Let m , ρ , and d be the per-operation costs of monitoring, records metadata, and policy evaluation, and let R be the one-time rebuild cost. Adaptation improves total time only if

$$T(\Delta - m - \rho - d) > R.$$

Equivalently, if $\Delta \leq m + \rho + d$, no phase length can make online adaptation profitable.

THEOREM 3 (ORACLE-HEADROOM LOWER BOUND). *Let $B_{\max}$ be the per-operation improvement of a zero-overhead oracle over an incumbent layout. Any online policy that retains the same incumbent and pays recurring overhead μ , nonnegative policy regret E , and nonnegative amortized restructuring cost R/T cannot outperform the incumbent when $B_{\max} \leq \mu$.*

PROOF. Relative to the incumbent, the online policy's best possible net improvement per operation is at most $B_{\max} - \mu - E - R/T$. Because $E \geq 0$ and $R/T \geq 0$, this quantity is nonpositive whenever $B_{\max} \leq \mu$. $\square$

For multiple phases, the same accounting yields a regret decomposition. If phase i has length T_i , oracle fanout f_i^* , selected fanout g_i , recurring overhead μ_i , and rebuild cost R_i , then

$$\text{Regret} = \sum_i T_i (c_{g_i} - c_{f_i^*}) + \sum_i T_i \mu_i + \sum_i R_i.$$

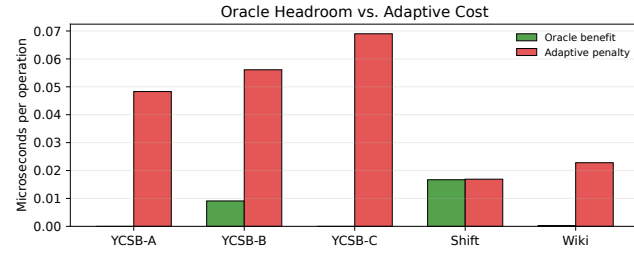


Figure 9: Measured oracle headroom versus adaptive penalty. In every tested case, the online cost is comparable to or larger than the available benefit.

The first term is decision error; the latter two remain even under perfect prediction. Our oracle measures the maximum amount available to pay all three terms, while the clean ablation estimates the recurring component.

Figure 9 compares measured headroom with measured penalty. On YCSB-A, the adaptive penalty is approximately $0.048 \mu\text{s/op}$; on YCSB-B and YCSB-C it is roughly 0.056 and $0.069 \mu\text{s/op}$. Current YCSB-A and YCSB-C have no positive static fanout headroom, while YCSB-B has about $0.009 \mu\text{s/op}$. On the shifting workload, the phase-oracle benefit and adaptive penalty are both about $0.017 \mu\text{s/op}$ before charging policy error or rebuild cost. On the Wikipedia trace, the static fanout-selection benefit is only about $0.0003 \mu\text{s/op}$, while the adaptive penalty is about $0.023 \mu\text{s/op}$.

This model does not say that adaptation is impossible. It says adaptation must be judged at the same time scale as the operation it improves. For in-memory B-trees with sub-microsecond lookups, software monitoring and rebuild decisions are expensive. For disk-resident trees, remote memory, or very long stable phases, the benefit term Δ may become large enough to dominate.

7 Discussion

7.1 Why This Negative Result Matters

The adaptive variants fail despite multiple policies, cost-aware rebuild gates, and workloads designed to expose skew. The failure is therefore not best understood as a single bad policy. It is a cost/benefit mismatch. The oracle rows show that choosing a better fanout can help. The overhead rows show that observing and acting on that opportunity can cost more than the opportunity itself.

7.2 When Adaptation Might Work

The break-even condition suggests several scenarios where online fanout adaptation could still be viable. First, disk-resident or remote-memory trees may have much larger fanout benefit because a height reduction can avoid expensive I/O. Second, workloads with very long stable phases can amortize rebuild cost. Third, hardware-assisted monitoring might reduce m , although it would not remove metadata and rebuild cost. Fourth, coarser adaptation at the granularity of entire indexes or partitions may be more practical than per-node or per-small-segment adaptation.

7.3 Practical Recommendation

For in-memory indexes at this scale, static tuning and coarse regional layout are safer than online fine-grained fanout adaptation. Practitioners should profile a representative workload and choose a static fanout or a small set of static regional fanouts. Researchers proposing adaptive index structures should include oracle baselines and component ablations; otherwise it is difficult to tell whether the design captures real headroom or merely shifts costs.

7.4 Design Implications

The measurements suggest a hierarchy of increasingly expensive responses to workload change. The first response should be an *opportunity test*: sample two layouts offline or during a rare maintenance window and disable the controller when their confidence intervals and time-per-operation difference show no usable headroom. The second response should be coarse partition-level tuning driven by an external phase signal, avoiding per-key heat metadata on the lookup path. Only when that oracle gap is large should the system enable continuous monitoring and segment reconstruction.

Three implementation changes follow directly from the ablation, and V2 implements concrete versions of all three: it removes the shadow records map, uses a fixed-size sampled monitor, and bulk-loads replacements in sorted order. The selected-regime results show that these changes improve the design but do not guarantee a win: they reduce μ and R , yet the best V2 setting still reaches only $0.881\text{--}0.931\times$ static throughput. More elaborate immutable snapshots or post-snapshot update logs remain possible, but the oracle-headroom theorem still rules out any design whose recurring cost exceeds the available layout benefit.

7.5 Threats to Validity

Platform and scale. All measurements use one laptop-class Intel CPU, one compiler toolchain, and in-memory integer keys. Different cache sizes, NUMA placement, allocators, or storage media can change both static fanout ordering and the relative cost of metadata. The 100K-record experiments are not a claim about billion-key or disk-resident indexes; they characterize the sub-microsecond regime in which the proposed control loop is most constrained.

Causal mechanism. We attribute costs to software components through controlled configuration ablations and elapsed time. The node-footprint analysis motivates cache sensitivity but is not presented as direct evidence of a particular microarchitectural event.

Workload coverage and noise. YCSB and shifting workloads use fixed seeds and controlled operation sequences, improving paired comparability but covering only one skew parameter and a limited set of phase lengths. Five trials leave broad intervals for several shifting rows; fixed benchmark order can also interact with frequency scaling and background activity. We disclose these intervals and avoid claims based on small adjacent differences. The Wikipedia data supplies a real popularity distribution, but mapping pageviews to point reads and updates does not reproduce a complete database workload.

Implementation boundary. The segmented architecture is one concrete design, not a lower bound over every possible adaptive B-tree. V1 uses a full shadow map and insertion-based rebuild; V2 removes that map, bounds its monitor, and bulk-loads from a sorted

export. V2 still performs a full-segment copy and uses a heuristic, uncalibrated cost gate. The concurrency experiment validates the static OLC tree; concurrent adaptive replacement and epoch reclamation are not implemented. The negative claim is therefore specific and testable: on these two implementation variants and these in-memory workloads, measured recurring cost exceeds or consumes the available oracle headroom. The analytical condition identifies what a different design must improve to overturn the result.

8 Related Work

B-trees and B+ trees are foundational ordered indexes [3, 8, 14]. Prior cache-conscious designs optimize node layout, prefetching, and memory access patterns [7, 28, 29]. Cache-oblivious trees study asymptotic transfer bounds without explicit cache parameters [4]. Our work differs by testing online fanout changes and measuring the cost of making the decision at runtime.

Architecture-sensitive and main-memory indexes explore complementary layout and hardware trade-offs. Examples include FAST, Masstree, HOT, and Wormhole [5, 18, 24, 38]. Concurrent and persistent B-tree variants such as the Bw-Tree, FPTree, and BzTree optimize indirection, synchronization, and recovery rather than online fanout selection [2, 23, 26]. The concurrency lineage also includes B-link trees, systematic locking analyses, and batched latch-free updates [13, 21, 31].

Adaptive and workload-aware indexing has a long history. Splay trees adjust to access patterns through rotations [32]. Database cracking and adaptive merging progressively refine physical organization based on queries [16, 17]. ART changes node representation based on local key density [22]. Learned indexes and learned variants adapt model structure or error bounds to data distributions [10, 12, 20]. These systems motivate our question, but they do not isolate online fanout adaptation costs in B-trees.

Robustness studies of adaptive indexing show that workload order and reorganization policy can dominate convergence cost [15, 30]. Dynamic and distribution-aware learned indexes provide further comparisons for adaptation under changing data and access patterns [11, 19, 33, 37], while controlled index benchmarks emphasize strong traditional baselines and build costs [25].

Self-tuning database systems select indexes, views, or configurations at much coarser time scales [6, 27, 34, 35]. Our result is consistent with that design lesson: adaptation is easier to justify when the cost is amortized over many operations. Fine-grained adaptation on a sub-microsecond access path is a much harsher setting.

9 Conclusion

We implemented adaptive B-tree fanout selection and expected skewed workloads to benefit. The data instead supports a diagnostic negative result. Static fanout choices and oracle phase-aware fanouts have measurable headroom, but the online path pays monitoring, metadata, policy, and rebuild costs that exceed the benefit on our in-memory workloads. A separate 1,740-run campaign confirms this result across selected seeds and sampling rates: even when a zero-cost oracle improves throughput, every paired interval for the best cost-aware adaptive configuration remains below static parity.

The resulting lesson is constructive: adaptive index papers should measure oracle headroom and adaptation overhead together. If the oracle cannot beat the baseline, no online method can win. If it wins only slightly, software adaptation must be nearly free. For the workloads tested here, fine-grained fanout adaptation is not yet a practical replacement for static tuning or coarse regional indexing.

Generative AI Disclosure

Large language models were used to assist with code scaffolding and boilerplate, and to improve the clarity, organization, and flow of the manuscript. The authors determined the research questions and system design, conducted and validated the experiments, verified the mathematical analysis and citations, interpreted the results, and take responsibility for the work.

References

- [1] Anastassia Ailamaki, David J. DeWitt, Mark D. Hill, and David A. Wood. 1999. DBMSs on a Modern Processor: Where Does Time Go?. In *Proceedings of the International Conference on Very Large Data Bases*. 266–277.
- [2] Joy Arulraj, Justin J. Levandoski, Umar Farooq Minhas, and Per-Åke Larson. 2018. BzTree: A High-Performance Latch-Free Range Index for Non-Volatile Memory. *Proceedings of the VLDB Endowment* 11, 5 (2018), 553–565. doi:10.1145/3187009.3164147
- [3] Rudolf Bayer and Edward McCreight. 1972. Organization and Maintenance of Large Ordered Indexes. *Acta Informatica* 1, 3 (1972), 173–189.
- [4] Michael A. Bender, Erik D. Demaine, and Martin Farach-Colton. 2000. Cache-Oblivious B-Trees. In *Proceedings of the IEEE Symposium on Foundations of Computer Science*. 399–409.
- [5] Robert Binna, Eva Zangerle, Martin Pichl, Günther Specht, and Viktor Leis. 2018. HOT: A Height Optimized Trie Index for Main-Memory Database Systems. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 521–534. doi:10.1145/3183713.3196896
- [6] Surajit Chaudhuri and Vivek Narasayya. 1998. AutoAdmin: What-If Index Analysis Utility. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 367–378.
- [7] Shimin Chen, Phillip B. Gibbons, Todd C. Mowry, and Gary Valentin. 2002. Fractal Prefetching B+-Trees: Optimizing Both Cache and Disk Performance. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 157–168.
- [8] Douglas Comer. 1979. The Ubiquitous B-Tree. *Comput. Surveys* 11, 2 (1979), 121–137.
- [9] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking Cloud Serving Systems with YCSB. In *Proceedings of the ACM Symposium on Cloud Computing*. 143–154.
- [10] Jialin Ding, Umar Farooq Minhas, Jia Yu, Chi Wang, Jaeyoung Do, Yinan Li, Hantian Zhang, Badrish Chandramouli, Johannes Gehrke, Donald Kossmann, and David Lomet. 2020. ALEX: An Updatable Adaptive Learned Index. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 969–984.
- [11] Paolo Ferragina and Giorgio Vinciguerra. 2020. The PGM-Index: A Fully-Dynamic Compressed Learned Index with Provable Worst-Case Bounds. *Proceedings of the VLDB Endowment* 13, 8 (2020), 1162–1175. doi:10.14778/3389133.3389135
- [12] Alex Galakatos, Michael Markovitch, Carsten Binnig, Rodrigo Fonseca, and Tim Kraska. 2019. FITing-Tree: A Data-Aware Index Structure. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 1189–1206.
- [13] Goetz Graefe. 2010. A Survey of B-Tree Locking Techniques. *ACM Transactions on Database Systems* 35, 3, Article 16 (2010), 26 pages. doi:10.1145/1806907.1806908
- [14] Goetz Graefe. 2011. Modern B-Tree Techniques. *Foundations and Trends in Databases* 3, 4 (2011), 203–402.
- [15] Felix Halim, Stratos Idreos, Panagiotis Karras, and Roland H. C. Yap. 2012. Stochastic Database Cracking: Towards Robust Adaptive Indexing in Main-Memory Column-Stores. *Proceedings of the VLDB Endowment* 5, 6 (2012), 502–513. doi:10.14778/2168651.2168652
- [16] Stratos Idreos, Martin L. Kersten, and Stefan Manegold. 2007. Database Cracking. In *Proceedings of the Conference on Innovative Data Systems Research*. 68–78.
- [17] Stratos Idreos, Martin L. Kersten, and Stefan Manegold. 2011. Adaptive Merging in Database Systems. *Proceedings of the VLDB Endowment* 4, 9 (2011), 586–597.
- [18] Changkyu Kim, Jatin Chhugani, Nadathur Satish, Eric Sedlar, Anthony D. Nguyen, Tim Kaldewey, Victor W. Lee, Scott A. Brandt, and Pradeep Dubey. 2010. FAST: Fast Architecture Sensitive Tree Search on Modern CPUs and GPUs. In *Proceedings of the ACM SIGMOD International Conference on Management of*

- Data. 339–350. doi:10.1145/1807167.1807206
- [19] Andreas Kipf, Ryan Marcus, Alexander van Renen, Mihail Stoian, Alfons Kemper, Tim Kraska, and Thomas Neumann. 2020. RadixSpline: A Single-Pass Learned Index. In *Proceedings of the International Workshop on Exploiting Artificial Intelligence Techniques for Data Management*. Article 5, 5 pages. doi:10.1145/3401071.3401659
- [20] Tim Kraska, Alex Beutel, Ed H. Chi, Jeffrey Dean, and Neoklis Polyzotis. 2018. The Case for Learned Index Structures. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 489–504.
- [21] Philip L. Lehman and S. Bing Yao. 1981. Efficient Locking for Concurrent Operations on B-Trees. *ACM Transactions on Database Systems* 6, 4 (1981), 650–670. doi:10.1145/319628.319663
- [22] Viktor Leis, Alfons Kemper, and Thomas Neumann. 2013. The Adaptive Radix Tree: ARTful Indexing for Main-Memory Databases. In *Proceedings of the IEEE International Conference on Data Engineering*. 38–49.
- [23] Justin J. Levandoski, David B. Lomet, and Sudipta Sengupta. 2013. The Bw-Tree: A B-Tree for New Hardware Platforms. In *Proceedings of the IEEE International Conference on Data Engineering*. 302–313.
- [24] Yandong Mao, Eddie Kohler, and Robert Tappan Morris. 2012. Cache Craftiness for Fast Multicore Key-Value Storage. In *Proceedings of the ACM European Conference on Computer Systems*. 183–196. doi:10.1145/2168836.2168855
- [25] Ryan Marcus, Mihail Stoian, Andreas Kipf, Sanchit Misra, Alexander van Renen, Alfons Kemper, Thomas Neumann, and Tim Kraska. 2020. Benchmarking Learned Indexes. *Proceedings of the VLDB Endowment* 14, 1 (2020), 1–13. doi:10.14778/3421424.3421425
- [26] Ismail Oukid, Daniel Booss, Adrien Lespinae, Wolfgang Lehner, Thomas Willhalm, and Giang Ngo. 2016. FPTree: A Hybrid SCM-DRAM Persistent and Concurrent B-Tree for Storage Class Memory. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 371–386. doi:10.1145/2882903.2915251
- [27] Andrew Pavlo, Gustavo Angulo, Joy Arulraj, Haibin Lin, Jiexi Lin, Lin Ma, Prashanth Menon, Todd C. Mowry, Matthew Perron, Ian Quah, Siddharth Santurkar, Anthony Tomic, Skye Toor, Dana Van Aken, Ziqi Wang, Yingjun Wu, Ran Xian, and Tieying Zhang. 2017. Self-Driving Database Management Systems. In *Proceedings of the Conference on Innovative Data Systems Research*.
- [28] Jun Rao and Kenneth A. Ross. 1999. Cache Conscious Indexing for Decision-Support in Main Memory. In *Proceedings of the International Conference on Very Large Data Bases*. 78–89.
- [29] Jun Rao and Kenneth A. Ross. 2000. Making B+-Trees Cache Conscious in Main Memory. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 475–486.
- [30] Martin Schuhknecht, Alekh Jindal, and Jens Dittrich. 2013. The Uncracked Pieces in Database Cracking. *Proceedings of the VLDB Endowment* 7, 2 (2013), 97–108.
- [31] Jason Sewall, Jatin Chhugani, Changkyu Kim, Nadathur Satish, and Pradeep Dubey. 2011. PALM: Parallel Architecture-Friendly Latch-Free Modifications to B+ Trees on Many-Core Processors. *Proceedings of the VLDB Endowment* 4, 11 (2011), 795–806.
- [32] Daniel D. Sleator and Robert E. Tarjan. 1985. Self-Adjusting Binary Search Trees. *J. ACM* 32, 3 (1985), 652–686.
- [33] Chuzhe Tang, Youyun Wang, Zhiyuan Dong, Gansen Hu, Zhaoguo Wang, Minjie Wang, and Haibo Chen. 2020. XIndex: A Scalable Learned Index for Multicore Data Storage. In *Proceedings of the ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. 308–320. doi:10.1145/3332466.3374547
- [34] Dana Van Aken, Andrew Pavlo, Geoffrey J. Gordon, and Bohan Zhang. 2017. Automatic Database Management System Tuning Through Large-Scale Machine Learning. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 1009–1024. doi:10.1145/3035918.3064029
- [35] Gerhard Weikum, Axel Moenkeberg, Christian Hasse, and Peter Zabback. 2002. Self-Tuning Database Technology and Information Services: From Wishful Thinking to Viable Engineering. *Proceedings of the VLDB Endowment* (2002), 20–31.
- [36] Wikimedia Foundation. 2026. Pageviews Analysis and Dumps. Accessed for workload trace construction.
- [37] Jiacheng Wu, Yong Zhang, Shimin Chen, Yu Chen, Jin Wang, and Chunxiao Xing. 2021. Updatable Learned Index with Precise Positions. *Proceedings of the VLDB Endowment* 14, 8 (2021), 1276–1288. doi:10.14778/3457390.3457393
- [38] Xingbo Wu, Fan Ni, and Song Jiang. 2019. Wormhole: A Fast Ordered Index for In-Memory Data Management. In *Proceedings of the ACM European Conference on Computer Systems*. Article 18, 21 pages. doi:10.1145/3302424.3303955
- [39] George Kingsley Zipf. 1949. *Human Behavior and the Principle of Least Effort*. Addison-Wesley.